Databases – Exercises Transactions

Vrije Universiteit Amsterdam

1. Consider the following schedule:

T1			W(X)			
T2		R(Y)				R(X)
T3	W(Z)			W(Y)		

Show that the following schedule is realisable with two-phase locking. For this purpose, insert all necessary lock and unlock actions.

Solution:

T1			X(X) W(X) U(X)			
T2		S(Y) R(Y)		S(X) U(Y)		R(X) U(X)
T3	X(Z) W(Z)				X(Y) W(Y) U(Y) U(Z)	

2. Let us revisit the schedule from the previous task:

T1			*start* W(X) *commit*			
T2		*start* R(Y)				R(X) *commit*
T3	*start* W(Z)			W(Y) *commit*		

The schedule now explicitly indicates the start and end of each transaction.

(a) Can this schedule be achieved using strict two phase locking?
Motivate your answer.

Solution: No, this schedule cannot be achieved using strict two phase locking. In strict two phase locking, every lock needs to be held until the end of the transaction. The transaction T2 needs a shared lock on Y before R(Y) and it needs to hold this shared lock until *commit*. Thus this lock will conflict with the exclusive lock on Y needed by transaction T3.

(b) Can this schedule be achieved using preclaiming two phase locking?
Motivate your answer.

Solution: No, this schedule cannot be achieved using preclaiming two phase locking. In preclaiming two phase locking, every lock needs to be declared immediately at the start of the transaction. The transaction T2 needs to declare a shared lock on X at the start of the transaction and this can only be released after R(X). As a consequence, this lock will conflict with the exclusive lock on X needed by transaction T1.

3. Consider the following schedule:

T1		*start* R(X)	W(Y) *commit*
T2	*start* R(X)	W(Y) *commit*	

(a) Can this schedule be achieved using strict two phase locking?
Motivate your answer.

> **Solution:** Yes, as follows:
>
T1		*start* S(X) R(X)	X(Y) W(Y) $\underbrace{\text{U(X) U(Y)}}_{commit}$
T2	*start* S(X) R(X)	X(Y) W(Y) $\underbrace{\text{U(X) U(Y)}}_{commit}$	
>
> Note that the locks are released only at the end of each transaction (on commit).

(b) Can this schedule be achieved using preclaiming two phase locking?
Motivate your answer.

> **Solution:** No, this schedule cannot be achieved using preclaiming two phase locking. The transaction T1 needs to declare an exclusive lock on Y at the start of the transaction and this can only be released after W(Y). As a consequence, this lock will conflict with (the lock needed for) W(Y) in the transaction T2.

4. Consider the following schedule:

T1	*start* W(X)		R(X) *commit*
T2		*start* R(X) *commit*	

(a) Can this schedule be achieved using strict two phase locking?
Motivate your answer.

> **Solution:** No, this schedule cannot be achieved using strict two phase locking. The transaction T1 needs an exclusive lock on X before W(X) and it needs to hold this lock until *commit*. Thus this lock will conflict with the lock on X needed by transaction T2.

(b) Can this schedule be achieved using preclaiming two phase locking?
Motivate your answer.

> **Solution:** Yes, as follows:
>
T1	$\underbrace{\text{S(X) X(X)}}_{start}$ W(X) UX(X)		R(X) US(X) *commit*
T2		$\underbrace{\text{S(X)}}_{start}$ R(X) US(X) *commit*	

> Note that transaction 1 obtains both a shared and an exclusive lock on X. Unlocking the exclusive lock while keeping the shared lock can also be understood as downgrading the exclusive lock to a shared lock.
>
> Likewise, in strict two phase locking it sometimes is necessary to upgrade locks from shared to exclusive, that is, obtaining an additional exclusive lock while already holding a shared lock on the same object.

5. Consider the following schedule:

T1			R(Z)	W(Y)					
T2		W(X)					R(Y)		
T3								R(Z)	
T4	W(Z)								R(X)
T5					W(Z)				
T6						R(X)			

Assume that transaction 4 is aborted/rolled back. Is this rollback cascading? What other transactions must be rolled back?

> **Solution:**
>
> 1. T4 is aborted
> 2. T1 has read Z written by T4, so **T1 needs to be aborted**
> 3. T3 does **not** need to be aborted (unless T5 is aborted), it has read Z written by T5
> 4. **T2 needs to be aborted** since it has read Y written by T1
> 5. **T6 needs to be aborted** since it has read X written by T2
>
> No other transaction needs to be aborted.

6. Consider the following schedule:

T1	R(X)	W(Y)			W(Z) *?*				
T2				W(X)		R(Y) ?			
T3			R(X)					R(A) *?*	
T4							R(Z)		W(A) *?*

Assume that the database management system uses backward oriented optimistic concurrency control. The question marks indicate the attempt of the transactions to commit. Determine the read sets and write sets for each transaction. Explain what happens with these transactions when they try to commit?

Solution: The read sets and write sets are:

1. RS(T1) = { X } and WS(T1) = { Y, Z }
2. RS(T2) = { Y } and WS(T2) = { X }
3. RS(T3) = { A, X } and WS(T3) = { }
4. RS(T4) = { Z } and WS(T4) = { A }

When these transactions commit:

1. when T1 commits:
   - **T1 is allowed to commit**, there is no transaction that has committed before.
2. when T2 commits:
   - T1 has committed after T2 started and RS(T2) ∩ WS(T1) = { Y }.
   - The intersection is not empty, so **T2 is aborted**.
3. when T3 commits:
   - T1 has committed after T3 started and RS(T3) ∩ WS(T1) = { }.
   - The intersection is empty, so **T3 is allowed to commit**.
4. when T4 commits:
   - T1 has committed before T4 has started
   - T3 has committed after T4 started and RS(T4) ∩ WS(T3) = { }.
   - So **T4 is allowed to commit**.